

Backend Schema

A PostgreSQL data model that keeps every startup workspace tenant-safe, traceable, and ready for AI-generated artifacts, financial scenarios, and execution work.

Version 1.0 | Made by DEX

Prepared from the Project VISTA source brief

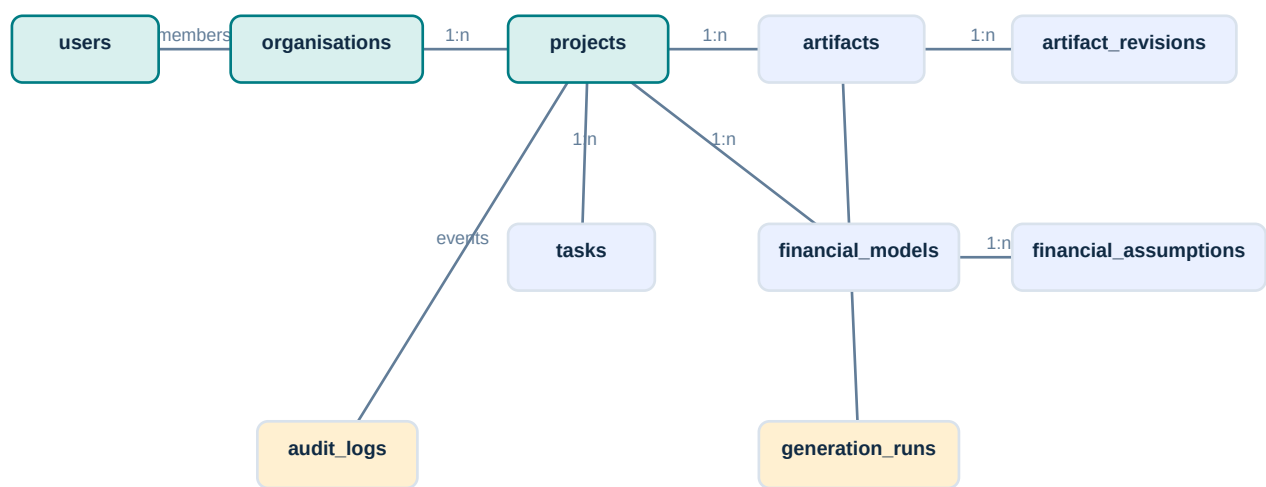
BACKEND SCHEMA

A coherent operating blueprint for an intelligent startup workspace.

1. Data Modelling Principles

- Use UUID primary keys and UTC timestamps on all business records.
- Store organisation_id on every tenant-owned table; apply row-level security or equivalent service-layer enforcement.
- Keep versioned content as immutable revisions once approved; never overwrite an approved version.
- Keep binary assets in object storage and persist only metadata and signed-access references in PostgreSQL.
- Use JSONB for flexible AI payloads and assumptions, but promote fields to typed columns when they drive queries or calculations.

2. Entity Relationship View



Core relationships. Supporting tables handle membership, sources, scenarios, files, notifications, and automation events.

3. Core Tables

Table	Purpose	Key fields
users	Authenticated identity profile.	id, email, display_name, avatar_url, created_at
organisations	Tenant boundary and billing owner.	id, name, slug, plan, created_at
organisation_members	User membership and role.	organisation_id, user_id, role, invited_at, joined_at
projects	A startup venture workspace.	id, organisation_id, name, stage, status, owner_id, context_json
project_members	Project-level access and role.	project_id, user_id, role
artifacts	Stable artifact identity such as business_plan or brand_kit.	id, project_id, type, title, current_revision_id, state
artifact_revisions	Editable/versioned artifact content.	id, artifact_id, version_no, content_json, content_text, created_by, approved_at
generation_runs	AI request, output, usage, and job state.	id, project_id, artifact_revision_id, kind, status, input_json, output_json, model, usage_json
tasks	Executable work item.	id, project_id, title, status, priority, owner_id, due_at, linked_artifact_id

Table	Purpose	Key fields
audit_logs	Security and business event trail.	id, organisation_id, actor_id, action, entity_type, entity_id, metadata_json, created_at

4. Specialist Tables

Domain	Tables	Notes
Research	research_jobs, research_sources, research_claims, competitors	Sources carry URL, publisher, retrieved_at, content hash, and trust state.
Brand	brand_kits, brand_assets	Brand kit is an artifact subtype; assets reference object storage.
Finance	financial_models, financial_scenarios, financial_assumptions, financial_outputs	Inputs are typed where possible; calculated output is reproducible from a formula version.
Automation	automation_rules, automation_runs, notifications	Use idempotency keys and a durable event record.
Files	files	Stores object key, media type, size, checksum, and ownership metadata.

5. State Models

Entity	States	Transition rule
Artifact	draft → in_review → approved → archived	Only a current draft/revision can be approved; approval actor and timestamp required.
Generation run	queued → running → succeeded failed cancelled	Retries create attempt records; failed output is retained for diagnostics.
Task	backlog → todo → in_progress → blocked done	Completion is auditable and may trigger an automation event.
Research claim	unverified → verified disputed deprecated	Verification captures reviewer and evidence reference.

6. Integrity, Indexing, and Security

- Foreign keys: projects.organisation_id; artifacts.project_id; revisions.artifact_id; tasks.project_id; all memberships reference their parent and user.
- Unique indexes: users.email; organisations.slug; (organisation_id, user_id) in organisation_members; (artifact_id, version_no) in artifact_revisions.
- Operational indexes: projects(organisation_id, updated_at); artifacts(project_id, state); tasks(project_id, status, due_at); generation_runs(project_id, status); audit_logs(organisation_id, created_at DESC).
- Use soft deletion only for user-visible business entities, with deleted_at and retention policy. Keep audit and billing events immutable.
- Encrypt credentials and provider tokens in a secrets vault, not in application tables. Record only non-secret provider references.

7. Example Approval Transaction

- Lock the artifact row and validate membership/role.
- Confirm the selected revision belongs to the artifact and is not superseded.
- Set the revision `approved_at` and `approved_by`; update artifact `current_revision_id` and state.
- Write `audit_logs` entry and publish an `artifact.approved` event via an outbox table.
- Worker consumes the event to create any user-authorised follow-up tasks or notifications.